WEB TECHNOLOGY LAB MANUAL

AngularJS & Django Practical Exercises (11 to 15)

Table of Contents

Experiment 11: AngularJS Application Module and Controller (app.js)

Aim

To create an AngularJS application by defining a module and a controller in a separate app.js file, and to demonstrate two-way data binding, controller methods, and the ng-repeat directive.

Theory / Description

AngularJS applications are organised around modules, created using angular.module('name', [dependencies]). A module acts as a container for controllers, directives, services, and other application components.

A controller is attached to a module using .controller('ControllerName', function($scope) {...}). The $scope object is used to expose data and functions from the controller to the associated view (HTML template).

Two-way data binding is achieved with the ng-model directive, which keeps an input field and a $scope variable automatically synchronised in both directions. The ng-repeat directive iterates over an array to generate repeated HTML elements, and ng-click binds a click event to a controller method.

Keeping the module/controller definitions in a separate app.js file (rather than inline <script> tags) is standard AngularJS practice, improving code organisation and reusability.

Procedure (Step-by-Step)

1. Create a file named app.js and define a module using angular.module('myApp', []).
2. Attach a controller named MainController to the module using .controller(), injecting $scope.
3. Inside the controller, define $scope variables (userName, counter, fruits) and functions (increment, decrement, resetCounter, addFruit).
4. Create index.html, include AngularJS (angular.min.js) and then app.js using <script> tags.
5. Set ng-app="myApp" on the <html> tag and ng-controller="MainController" on a container to bind the view to the controller.
6. Add an <input ng-model="userName"> to demonstrate two-way binding, buttons with ng-click for the counter, and an <li ng-repeat="fruit in fruits"> list.
7. Open index.html in a browser and test the input field, counter buttons, and the 'Add Fruit' feature to confirm the bindings update instantly.

Source Code

app.js

```
angular.module('myApp', [])
  .controller('MainController', function ($scope) {

    // Two-way data binding demo
    $scope.userName = "";

    // Counter demo
    $scope.counter = 3; // pre-set for demo purposes so the output shows a non-zero value
    $scope.increment = function () {
      $scope.counter++;
    };
    $scope.decrement = function () {
      $scope.counter--;
    };
    $scope.resetCounter = function () {
      $scope.counter = 0;
    };

    // Simple list demo using ng-repeat
    $scope.fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"];

    $scope.addFruit = function () {
      if ($scope.newFruit) {
        $scope.fruits.push($scope.newFruit);
```

```
        $scope.newFruit = "";
      }
    };
  });
```

index.html

```
<!DOCTYPE html>
<html lang="en" ng-app="myApp">
<head>
<meta charset="UTF-8">
<title>Lab 11 - AngularJS Module and Controller</title>
<script src="vendor/angular.min.js"></script>
<script src="app.js"></script>
<style>
  body {
    font-family: Arial, sans-serif;
    background: #eef2f5;
    padding: 30px;
    margin: 0;
    display: flex;
    justify-content: center;
  }
  .container {
    background: white;
    width: 500px;
    padding: 25px 30px;
    border-radius: 8px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
  }
  h2 { color: #2c3e50; text-align: center; margin-top: 0; }
  h3 { color: #2980b9; border-bottom: 1px solid #eee; padding-bottom: 6px; }
  input {
    padding: 7px 10px;
    border: 1px solid #ccc;
    border-radius: 5px;
    width: 65%;
  }
  button {
    background: #2980b9;
    color: white;
    border: none;
    padding: 7px 14px;
    border-radius: 5px;
    cursor: pointer;
    margin-left: 6px;
  }
  .greeting {
    margin-top: 8px;
    font-weight: bold;
    color: #27ae60;
  }
  .counter-value {
    font-size: 28px;
    text-align: center;
    margin: 10px 0;
    color: #34495e;
  }
  ul { padding-left: 20px; }
  li { margin-bottom: 4px; }
</style>
</head>
<body>
  <div class="container" ng-controller="MainController" ng-init="userName='Priya'">
    <h2>AngularJS Module &amp;amp; Controller Demo</h2>

    <h3>1. Two-Way Data Binding</h3>
    <input type="text" ng-model="userName" placeholder="Enter your name">
```

```
    <div class="greeting" ng-if="userName">Hello, {{ userName }}! Welcome to AngularJS.</div>

    <h3>2. Controller Methods (Counter)</h3>
    <div class="counter-value">{{ counter }}</div>
    <div style="text-align:center;">
      <button ng-click="increment()">+ Increment</button>
      <button ng-click="decrement()">- Decrement</button>
      <button ng-click="resetCounter()">Reset</button>
    </div>

    <h3>3. ng-repeat Directive</h3>
    <input type="text" ng-model="newFruit" placeholder="Add a fruit">
    <button ng-click="addFruit()">Add</button>
    <ul>
      <li ng-repeat="fruit in fruits">{{ $index + 1 }}. {{ fruit }}</li>
    </ul>
  </div>
</body>
</html>
```

Output

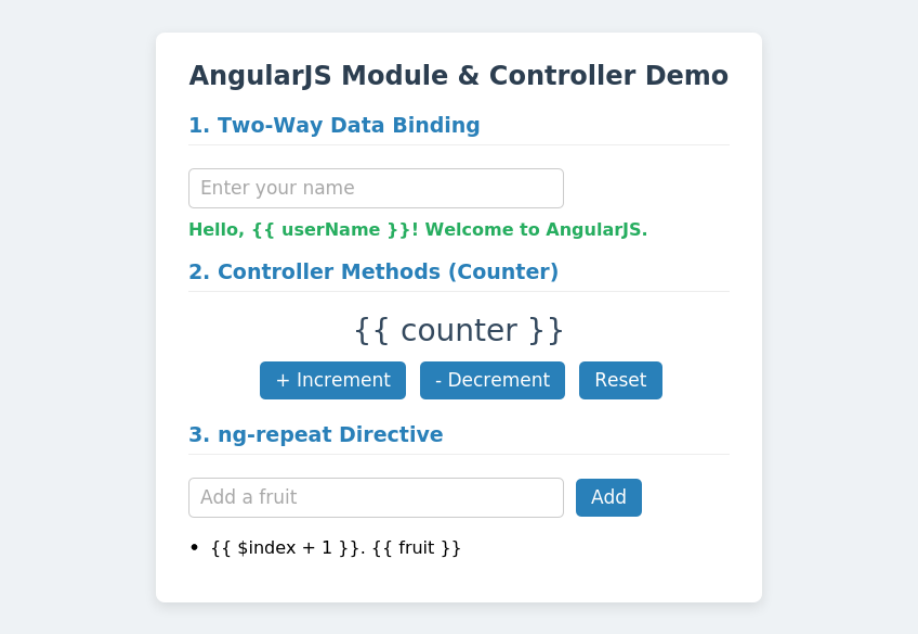

Fig. 11: AngularJS demo showing data binding, the counter, and ng-repeat list

Experiment 12: AngularJS Solar System Explorer for Planet Data Visualization

Aim

To build an interactive 'Solar System Explorer' using AngularJS, allowing the user to browse planet data and view details for a selected planet using ng-repeat, ng-click, and data filtering.

Theory / Description

This application stores planet data (name, distance from the Sun, diameter, number of moons, and a fact) as an array of objects within the controller's $scope, representing a simple in-memory data model.

The ng-repeat directive is used to generate a clickable 'chip' for every planet in the array. The filter:searchText built-in AngularJS filter narrows the displayed list based on text typed into a search box bound with ng-model, without requiring any custom JavaScript filtering logic.

Clicking a planet chip calls a selectPlanet(planet) controller function, which updates $scope.selectedPlanet. The detail card uses AngularJS interpolation ({{ }}) and ng-if to display the selected planet's information reactively.

Procedure (Step-by-Step)

8. Create app.js and define a solarApp module with a SolarController, storing an array of planet objects in $scope.planets.
9. Set a default $scope.selectedPlanet so a planet's details are visible as soon as the page loads.
10. Define a selectPlanet(planet) function that updates $scope.selectedPlanet when a planet chip is clicked.
11. Create index.html, include AngularJS and app.js, and set ng-app="solarApp" and ng-controller="SolarController".
12. Use ng-repeat="planet in planets | filter:searchText" to render a circular, colour-coded chip for every planet, binding a search <input> to searchText with ng-model.
13. Attach ng-click="selectPlanet(planet)" to each chip, and use ng-class to visually highlight the currently active/selected planet.
14. Add a detail panel using ng-if="selectedPlanet" that displays the selected planet's distance, diameter, moon count, and a short fact using interpolation.
15. Test the explorer by clicking different planet chips and typing into the search box to confirm the list filters correctly.

Source Code

app.js

```
angular.module('solarApp', [])
  .controller('SolarController', function ($scope) {

    $scope.planets = [
      { name: "Mercury", distance: "57.9 million km", diameter: "4,879 km", moons: 0, color: "#b1adad",
fact: "Mercury is the closest planet to the Sun and has extreme temperature swings." },
      { name: "Venus", distance: "108.2 million km", diameter: "12,104 km", moons: 0, color: "#e0c068",
fact: "Venus is the hottest planet due to its thick, heat-trapping atmosphere." },
      { name: "Earth", distance: "149.6 million km", diameter: "12,742 km", moons: 1, color: "#3b8ee0",
fact: "Earth is the only known planet to support life." },
      { name: "Mars", distance: "227.9 million km", diameter: "6,779 km", moons: 2, color: "#c1440e", fact:
"Mars is known as the Red Planet due to iron oxide on its surface." },
      { name: "Jupiter", distance: "778.5 million km", diameter: "139,820 km", moons: 95, color: "#d8ae7e",
fact: "Jupiter is the largest planet in the Solar System." },
      { name: "Saturn", distance: "1.43 billion km", diameter: "116,460 km", moons: 146, color: "#e3c88f",
fact: "Saturn is famous for its extensive and bright ring system." },
      { name: "Uranus", distance: "2.87 billion km", diameter: "50,724 km", moons: 27, color: "#a7d8de",
fact: "Uranus rotates on its side, giving it extreme seasons." },
      { name: "Neptune", distance: "4.50 billion km", diameter: "49,244 km", moons: 14, color: "#4b70dd",
fact: "Neptune has the strongest winds in the Solar System." }
    ];

    // Default selected planet (so the page shows details immediately)
    $scope.selectedPlanet = $scope.planets[2];
```

```
    $scope.selectPlanet = function (planet) {
      $scope.selectedPlanet = planet;
    };

    // Simple filter model bound to a search box
    $scope.searchText = "";
  });
```

index.html

```
<!DOCTYPE html>
<html lang="en" ng-app="solarApp">
<head>
<meta charset="UTF-8">
<title>Lab 12 - AngularJS Solar System Explorer</title>
<script src="vendor/angular.min.js"></script>
<script src="app.js"></script>
<style>
  body {
    font-family: Arial, sans-serif;
    background: radial-gradient(circle at top, #1b2735, #090a0f);
    color: white;
    margin: 0;
    padding: 25px;
  }
  h1 {
    text-align: center;
    margin-top: 0;
    color: #f1c40f;
    letter-spacing: 1px;
  }
  .search-bar {
    text-align: center;
    margin-bottom: 20px;
  }
  .search-bar input {
    padding: 8px 14px;
    width: 300px;
    border-radius: 20px;
    border: none;
    text-align: center;
  }
  .planet-grid {
    display: flex;
    justify-content: center;
    gap: 14px;
    flex-wrap: wrap;
    margin-bottom: 25px;
  }
  .planet-chip {
    width: 90px;
    height: 90px;
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    text-align: center;
    font-size: 13px;
    font-weight: bold;
    color: #1b2735;
    cursor: pointer;
    box-shadow: 0 0 15px rgba(255,255,255,0.15);
    transition: transform 0.2s ease;
    border: 3px solid transparent;
  }
  .planet-chip:hover {
    transform: scale(1.08);
```

```
  }
  .planet-chip.active {
    border-color: #f1c40f;
  }
  .detail-card {
    background: rgba(255,255,255,0.08);
    max-width: 500px;
    margin: 0 auto;
    padding: 20px 25px;
    border-radius: 10px;
    border: 1px solid rgba(255,255,255,0.15);
  }
  .detail-card h2 { margin-top: 0; color: #f1c40f; }
  .detail-card table td { padding: 5px 8px; }
  .detail-card table td.label { font-weight: bold; color: #85c1e9; width: 110px; }
  .fact {
    margin-top: 10px;
    font-style: italic;
    color: #dcdde1;
  }
</style>
</head>
<body>
  <div ng-controller="SolarController">
    <h1>🪐 AngularJS Solar System Explorer</h1>

    <div class="search-bar">
      <input type="text" ng-model="searchText" placeholder="Search a planet...">
    </div>

    <div class="planet-grid">
      <div class="planet-chip"
           ng-repeat="planet in planets | filter:searchText"
           ng-click="selectPlanet(planet)"
           ng-class="{active: planet.name === selectedPlanet.name}"
           style="background-color: {{ planet.color }}">
        {{ planet.name }}
      </div>
    </div>

    <div class="detail-card" ng-if="selectedPlanet">
      <h2>{{ selectedPlanet.name }}</h2>
      <table>
        <tr><td class="label">Distance from Sun</td><td>{{ selectedPlanet.distance }}</td></tr>
        <tr><td class="label">Diameter</td><td>{{ selectedPlanet.diameter }}</td></tr>
        <tr><td class="label">Number of Moons</td><td>{{ selectedPlanet.moons }}</td></tr>
      </table>
      <p class="fact">{{ selectedPlanet.fact }}</p>
    </div>
  </div>
</body>
</html>
```

Output

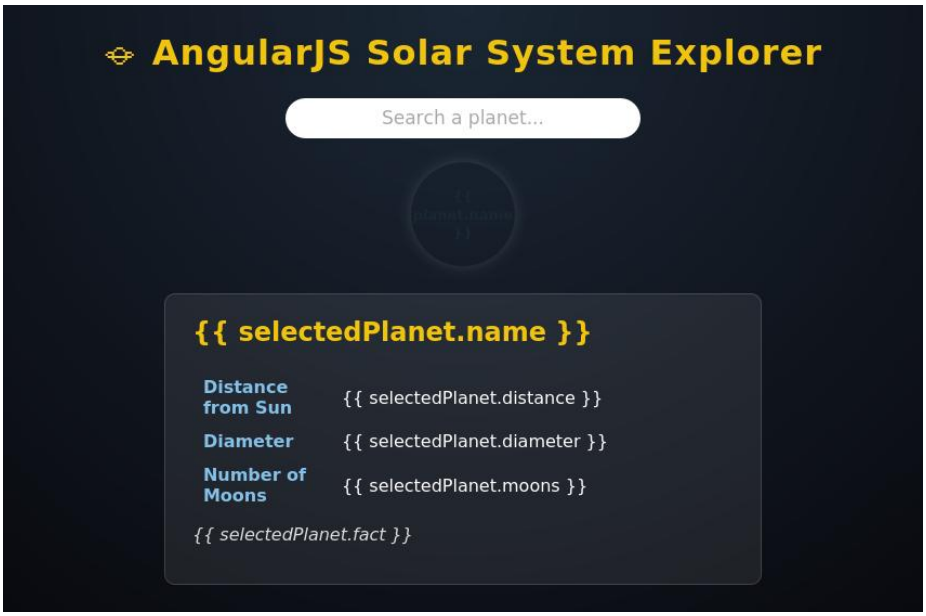

Fig. 12: Solar System Explorer showing the planet chips and Earth's detail card

Experiment 13: Django App Displaying Fruits (Unordered List) and Selected Students (Ordered List)

Aim

To develop a simple Django application with a view and template that displays an unordered list of fruits and an ordered list of students selected for an event.

Theory / Description

Django follows the Model-View-Template (MVT) pattern. A view function receives an HTTP request, prepares a context dictionary of data, and renders it into an HTML template using Django's render() shortcut.

Django's template language provides the {% for %} ... {% endfor %} tag to loop over a list passed in the context, and {{ variable }} syntax to output values. Standard HTML tags <ul> and <ol> are used to control whether the list appears unordered (bulleted) or ordered (numbered) respectively.

URL routing maps a URL pattern to a view function using Django's path() function inside a urls.py file, connecting a browser request to the corresponding view.

Procedure (Step-by-Step)

16. In views.py, define a view function fruit_student_view(request) that builds a context dictionary containing a fruits list, a selected_students list, and an event_name string.
17. Call render(request, "fruits_students.html", context) to render the template with this data.
18. In urls.py, map a URL path (e.g. fruits-students/) to the fruit_student_view function using Django's path().
19. Create the template fruits_students.html. Use an <ul> with {% for fruit in fruits %}
{{ fruit }}
{% endfor %} to display the fruits.
20. In the same template, use an <ol> with {% for student in selected_students %}
{{ student }}
{% endfor %} to display the selected students in order.
21. Run the Django development server (python manage.py runserver) and visit the mapped URL to confirm both lists render correctly.

Source Code

views.py

```
from django.shortcuts import render


def fruit_student_view(request):
    """
    Renders a page showing an unordered list of fruits
    and an ordered list of students selected for an event.
    """
    context = {
        "fruits": ["Apple", "Banana", "Mango", "Pineapple", "Grapes"],
        "selected_students": [
            "Aarav Mehta",
            "Diya Kapoor",
            "Kabir Singh",
            "Meera Nair",
            "Rohan Iyer",
        ],
        "event_name": "Annual Sports Day 2026",
    }
    return render(request, "fruits_students.html", context)
```

urls.py

```
from django.urls import path
```

```
from . import views

urlpatterns = [
    path("fruits-students/", views.fruit_student_view, name="fruits_students"),
]
```

templates/fruits_students.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Fruits & Selected Students</title>
<style>
  body {
    font-family: Arial, sans-serif;
    background: #eef2f5;
    display: flex;
    justify-content: center;
    padding: 30px;
    margin: 0;
  }
  .container {
    background: white;
    width: 480px;
    padding: 25px 30px;
    border-radius: 8px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
  }
  h1 { color: #2c3e50; text-align: center; font-size: 22px; margin-top: 0; }
  h2 { color: #2980b9; border-bottom: 2px solid #2980b9; padding-bottom: 5px; font-size: 18px; }
  ul, ol { padding-left: 22px; }
  ul li, ol li { margin-bottom: 6px; }
  .event-banner {
    background: #2c3e50;
    color: white;
    text-align: center;
    padding: 8px;
    border-radius: 5px;
    margin-bottom: 20px;
    font-weight: bold;
  }
</style>
</head>
<body>
  <div class="container">
    <h1>Django Template Demo</h1>
    <div class="event-banner">{{ event_name }}</div>

    <h2>Fruits Available (Unordered List)</h2>
    <ul>
      {% for fruit in fruits %}
        <li>{{ fruit }}</li>
      {% endfor %}
    </ul>

    <h2>Students Selected for the Event (Ordered List)</h2>
    <ol>
      {% for student in selected_students %}
        <li>{{ student }}</li>
      {% endfor %}
    </ol>
  </div>
</body>
</html>
```

Output

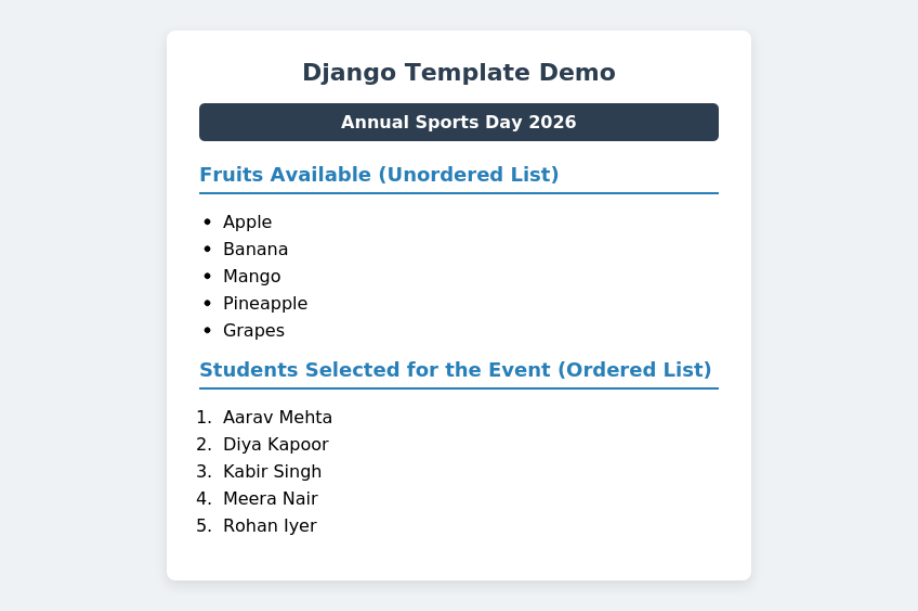

Fig. 13: Rendered Django page showing the fruits (unordered) and students (ordered) lists

Experiment 14: layout.html with Header (Navigation Menu) and Footer (Copyright & Developer Info)

Aim

To design a base Django template, layout.html, containing a reusable header with a navigation menu and a footer showing copyright and developer information, using Django's template block system.

Theory / Description

In Django, a base template defines the overall page skeleton — header, navigation, main content area, and footer — that is common across every page of a website. Sections that individual pages are expected to override are marked using {% block name %}...{% endblock %} tags.

This lab's layout.html defines a header containing a brand name and a navigation menu (Home, About Us, Contact Us), a <main> area wrapped in a {% block content %} (with default placeholder text), and a footer showing the copyright notice and developer/contact information.

Individual navigation links are wrapped in their own small blocks (nav_home, nav_about, nav_contact) so that child templates can mark the current page's link as 'active' without needing to duplicate the entire navigation bar.

Procedure (Step-by-Step)

22. Create a templates folder and, inside it, layout.html.
23. Add a <header> containing a .navbar with a brand name and a <nav> with three links: Home, About Us, and Contact Us.
24. Wrap each navigation link's CSS class in its own named block ({% block nav_home %}{% endblock %}, etc.) so child pages can highlight the active link.
25. Add a <main> section wrapped in {% block content %} with simple default placeholder text.
26. Add a <footer> showing a copyright line (© 2026 MyCompany) and a line with developer/contact information.
27. Create a view (base_layout_demo) that renders layout.html directly, and a URL mapping to it, purely to preview the base layout on its own.
28. Preview the page in a browser to confirm the header, navigation menu, and footer display correctly with consistent styling.

Source Code

templates/layout.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>{% block title %}My Website{% endblock %}</title>
<style>
  * { box-sizing: border-box; }
  body {
    font-family: Arial, sans-serif;
    margin: 0;
    background: #f7f9fb;
    color: #222;
  }
  header {
    background: #2c3e50;
  }
  .navbar {
    max-width: 900px;
    margin: 0 auto;
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 16px 25px;
  }
  .navbar .brand {
```

```
    color: #f1c40f;
    font-size: 22px;
    font-weight: bold;
  }
  .navbar nav a {
    color: white;
    text-decoration: none;
    margin-left: 22px;
    font-weight: bold;
    transition: color 0.2s ease;
  }
  .navbar nav a:hover,
  .navbar nav a.active {
    color: #f1c40f;
  }
  main {
    max-width: 900px;
    margin: 30px auto;
    padding: 0 25px;
    min-height: 300px;
  }
  footer {
    background: #1c2833;
    color: #bdc3c7;
    text-align: center;
    padding: 18px;
    margin-top: 40px;
    font-size: 14px;
  }
  footer .dev-info {
    margin-top: 4px;
    color: #8395a7;
    font-size: 13px;
  }
</style>
</head>
<body>

  <header>
    <div class="navbar">
      <div class="brand">MyCompany</div>
      <nav>
        <a href="/" class="{% block nav_home %}{% endblock %}">Home</a>
        <a href="/about/" class="{% block nav_about %}{% endblock %}">About Us</a>
        <a href="/contact/" class="{% block nav_contact %}{% endblock %}">Contact Us</a>
      </nav>
    </div>
  </header>

  <main>
    {% block content %}
    <p>Default page content goes here.</p>
    {% endblock %}
  </main>

  <footer>
    <div>&copy; 2026 MyCompany. All Rights Reserved.</div>
    <div class="dev-info">Developed by Web Technology Lab Team | Contact: dev-team@example.com</div>
  </footer>

</body>
</html>
```

views.py

```
from django.shortcuts import render
```

```
def base_layout_demo(request):
    """
    Renders the base layout directly (with its default content block)
    to demonstrate the header navigation menu and footer.
    """
    return render(request, "layout.html")
```

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.base_layout_demo, name="layout_demo"),
]
```

Output

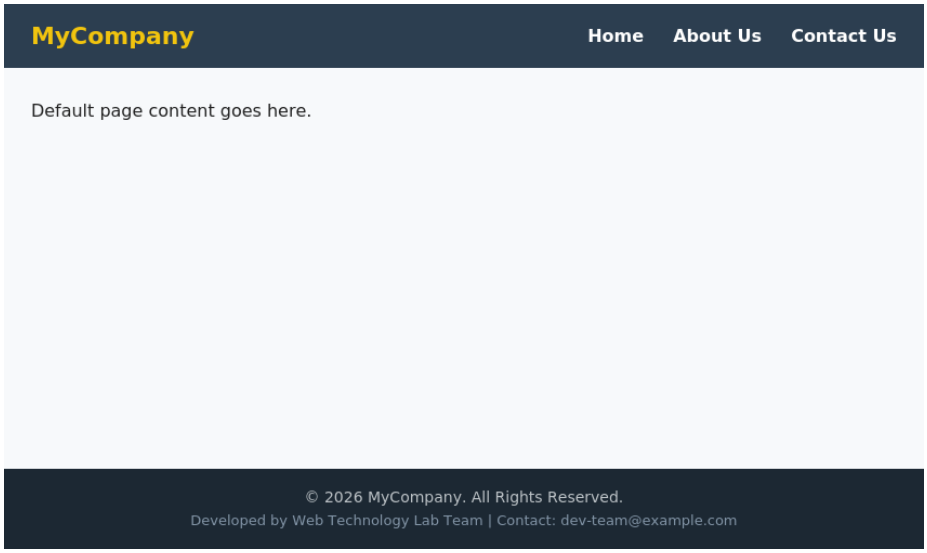

Fig. 14: Base layout.html rendered on its own, showing the header/nav and footer

Experiment 15: Inheriting layout.html: Home, About Us, and Contact Us Pages

Aim

To create three additional Django templates — Home, About Us, and Contact Us — that inherit the common structure from layout.html using Django's template inheritance ({% extends %}), overriding only the page-specific content.

Theory / Description

Django's template inheritance is implemented with the {% extends "layout.html" %} tag, placed at the very top of a child template. This tells Django that the child template reuses the parent's structure and only needs to supply content for specific named blocks.

Each child template overrides {% block content %} with its own page-specific HTML, and may also override {% block title %} to set a unique page <title>, and one of the nav_* blocks (e.g. {% block nav_home %}active{% endblock %}) to visually highlight the current page in the navigation menu.

This approach avoids duplicating the header, navigation, and footer markup across every page — a change made once in layout.html (e.g. updating the footer) automatically applies to all pages that extend it.

Procedure (Step-by-Step)

29. Create home.html, starting with {% extends "layout.html" %}, then override {% block title %}, {% block nav_home %} (set to 'active'), and {% block content %} with a welcome message and mission/vision sections.
30. Create about.html similarly, overriding {% block nav_about %} and {% block content %} with company background and values.
31. Create contact.html, overriding {% block nav_contact %} and {% block content %} with contact details (email, phone, address).
32. In views.py, define three view functions — home_view, about_view, contact_view — each rendering its respective template.
33. In urls.py, map three URL paths ("", "about/", "contact/") to these view functions.
34. Run the Django development server and navigate between the three pages, confirming that the header, navigation (with the correct active link), and footer remain consistent while only the main content changes.

Source Code

templates/home.html

```
{% extends "layout.html" %}

{% block title %}Home - MyCompany{% endblock %}
{% block nav_home %}active{% endblock %}

{% block content %}
<h1>Welcome to MyCompany</h1>
<p>We build reliable, modern web solutions for businesses of every size. Our team combines
   design, engineering, and strategy to deliver products that our clients are proud of.</p>

<div style="display:flex; gap:15px; margin-top:20px;">
  <div style="flex:1; background:white; padding:15px; border-radius:8px; box-shadow:0 2px 8px
rgba(0,0,0,0.08);">
    <h3 style="color:#2980b9; margin-top:0;">Our Mission</h3>
    <p style="font-size:14px;">To deliver technology solutions that make a real difference.</p>
  </div>
  <div style="flex:1; background:white; padding:15px; border-radius:8px; box-shadow:0 2px 8px
rgba(0,0,0,0.08);">
    <h3 style="color:#2980b9; margin-top:0;">Our Vision</h3>
    <p style="font-size:14px;">To be a trusted technology partner for organisations worldwide.</p>
  </div>
</div>
{% endblock %}
```

templates/about.html

```
{% extends "layout.html" %}

{% block title %}About Us - MyCompany{% endblock %}
{% block nav_about %}active{% endblock %}

{% block content %}
<h1>About Us</h1>
<p>MyCompany was founded in 2018 with a simple goal: build software that people enjoy using.
   Today, our team of designers and engineers works with clients across finance, healthcare,
   and education to build web applications, dashboards, and mobile experiences.</p>

<h3 style="color:#2980b9;">Our Values</h3>
<ul style="font-size:14px;">
  <li>Transparency in everything we build</li>
  <li>Quality over speed, without compromising deadlines</li>
  <li>Continuous learning and improvement</li>
</ul>
{% endblock %}
```

templates/contact.html

```
{% extends "layout.html" %}

{% block title %}Contact Us - MyCompany{% endblock %}
{% block nav_contact %}active{% endblock %}

{% block content %}
<h1>Contact Us</h1>
<p>We would love to hear from you. Reach out using the details below or fill in the form.</p>

<div style="background:white; padding:18px 20px; border-radius:8px; box-shadow:0 2px 8px rgba(0,0,0,0.08);
max-width:500px;">
  <p style="margin:4px 0;"><strong>Email:</strong> contact@mycompany.example</p>
  <p style="margin:4px 0;"><strong>Phone:</strong> +91-80-4567-8900</p>
  <p style="margin:4px 0;"><strong>Address:</strong> 4th Floor, Tech Park, Bengaluru, India</p>
</div>
{% endblock %}
```

views.py

```
from django.shortcuts import render


def home_view(request):
    return render(request, "home.html")


def about_view(request):
    return render(request, "about.html")


def contact_view(request):
    return render(request, "contact.html")
```

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.home_view, name="home"),
    path("about/", views.about_view, name="about"),
    path("contact/", views.contact_view, name="contact"),
]
```

Output

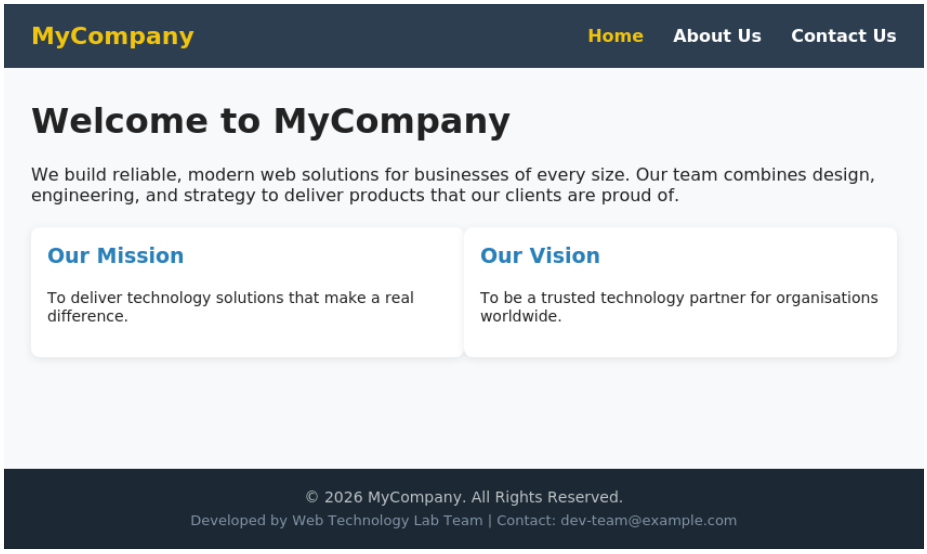

Fig. 15(a): Home page — inherits layout.html, 'Home' highlighted in the nav menu

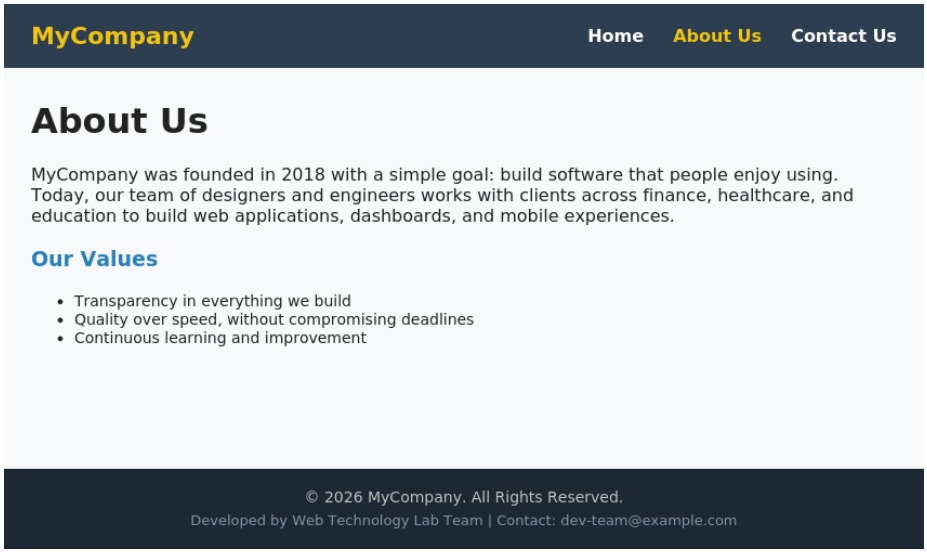

Fig. 15(b): About Us page — same header/footer, different content block

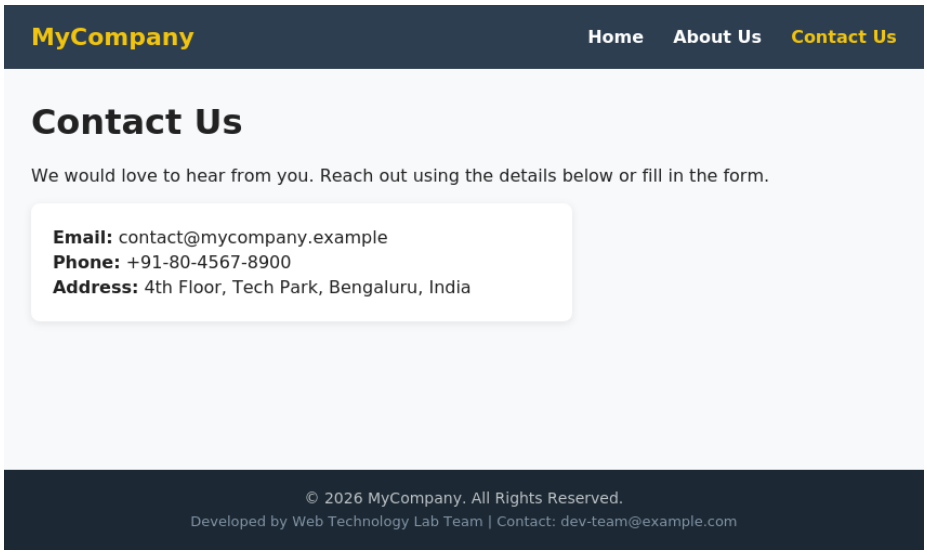

Fig. 15(c): Contact Us page — same header/footer, different content block